

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: شایان عباسی

شماره دانشجویی: 04121040768022

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

پروژه 4: نرم افزار تحلیل پاسخ سیستم های مرتبه اول و دوم

موضوع

ابزار آموزشی کنترل

ورودی

K

ζ

ω_n

خروجی

- نمودار پله
- زمان نشست
- فرجهش
- زمان صعود

GUI

کاربر پارامترها را وارد کند.

کلاس ها

TransferFunction

StepResponse

Plotter

Analyzer

1. کتابخانه ها

```
import numpy as np
```

کتابخانه numpy را با نام مستعار np وارد کردم. از این کتابخانه برای محاسبات عددی مانند ساخت بردار زمان با linspace و عملیات نمایی و مثلثاتی روی آرایه ها در محاسبه پاسخ پله استفاده می شود.

```
import matplotlib.pyplot as plt
```

کتابخانه matplotlib را برای رسم نمودار پاسخ پله سیستم وارد کردم. از این کتابخانه برای ساخت figure و axes و رسم منحنی خروجی همراه با خطوط راهنما استفاده می شود.

```
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
```

این کلاس امکان جاسازی نمودار matplotlib داخل پنجره tkinter را فراهم می کند. با استفاده از آن نمودار مستقیماً در سمت راست پنجره کنار پنل ورودی ها نمایش داده می شود.

```
import tkinter as tk
```

کتابخانه tkinter را برای ساخت رابط گرافیکی برنامه وارد کردم. از این کتابخانه برای ساخت پنجره اصلی، Label و Frame، Entry، Button استفاده می شود.

```
from tkinter import ttk, messagebox
```

ماژول ttk برای استفاده از Separator که خطوط جداکننده بین بخش های پنل را رسم می کند به کار رفته. ماژول messagebox برای نمایش پیام خطا وقتی کاربر مقدار نامعتبر وارد می کند استفاده می شود.

۲. کلاس TransferFunction (تابع تبدیل)

```
class TransferFunction:
```

```
    def __init__(self, K, order, zeta=None, wn=None, tau=None):
```

```
        self.K = K
```

```
        self.order = order
```

```
self.zeta = zeta
```

```
self.wn = wn
```

```
self.tau = tau
```

این کلاس را برای مدل‌سازی تابع تبدیل سیستم طراحی کردم. K بهره سیستم و order مرتبه آن (۱ یا ۲) است. برای سیستم مرتبه اول τ ثابت زمانی استفاده می‌شود و برای سیستم مرتبه دوم zeta نسبت میرایی و wn فرکانس طبیعی به کار می‌روند.

متد `get_description` (نمایش معادله تابع تبدیل)

```
def get_description(self):
    if self.order == 1:
        return f"G(s) = {self.K} / ({self.tau}s + 1)"
    else:
        a = self.K * self.wn ** 2
        b = 2 * self.zeta * self.wn
        c = self.wn ** 2
        return f"G(s) = {a:.2f} / (s^2 + {b:.2f}s + {c:.2f})"
```

این متد معادله تابع تبدیل را به صورت رشته متنی برمی‌گرداند تا روی عنوان نمودار نمایش داده شود. برای مرتبه اول فرم استاندارد $K/(ts+1)$ و برای مرتبه دوم فرم استاندارد بر اساس ضرایب wn و zeta محاسبه و نمایش داده می‌شود.

متد `get_system_type` (تشخیص نوع سیستم)

```
def get_system_type(self):
    if self.order == 1:
        return "First Order System"
    if self.zeta == 0:
        return "Undamped"
    elif self.zeta < 1:
        return "Underdamped"
    elif self.zeta == 1:
        return "Critically Damped"
    else:
        return "Overdamped"
```

این متد بر اساس مرتبه و مقدار ζ نوع رفتار سیستم را تشخیص می‌دهد. برای سیستم مرتبه دوم چهار حالت ممکن است: بدون میرایی، کم‌میرا، میرایی بحرانی و بیش‌میرا. این اطلاعات در کنار نتایج تحلیل به کاربر نمایش داده می‌شود.

۳. کلاس StepResponse (محاسبه پاسخ پله)

```
class StepResponse:
    def __init__(self, tf):
        self.tf = tf
```

این کلاس را برای محاسبه پاسخ پله سیستم بر اساس تابع تبدیل طراحی کردم. یک شیء TransferFunction را دریافت می‌کند و معادله تحلیلی پاسخ پله را برای آن حل می‌کند.

متد compute (محاسبه بردار زمان و پاسخ)

```
def compute(self):
    if tf.order == 1:
        t_end = 5 * tf.tau
    else:
        if tf.zeta >= 1:
            t_end = 10 / (tf.zeta * tf.wn)
        else:
            t_end = max(20/tf.wn, 10/(tf.zeta*tf.wn+0.001))
    t = np.linspace(0, t_end, 1000)
    if tf.order == 1:
        y = tf.K * (1 - np.exp(-t / tf.tau))
    else:
        if z < 1:
            wd = w * np.sqrt(1 - z**2)
            y = K*(1 - np.exp(-z*w*t)*(np.cos(wd*t)+phi*np.sin(wd*t)))
    return t, y
```

این متد مهم‌ترین بخش محاسباتی برنامه است. ابتدا بازه زمانی مناسب t_{end} بر اساس پارامترهای سیستم تخمین زده می‌شود تا کل رفتار گذرا در نمودار دیده شود. برای سیستم مرتبه اول پاسخ نمایی استاندارد محاسبه می‌شود.

برای سیستم مرتبه دوم بسته به مقدار ζ چهار فرمول تحلیلی متفاوت (بدون میرایی، کم میرا، بحرانی، بیش میرا) استفاده می‌شود تا پاسخ دقیق هر حالت محاسبه گردد.

۴. کلاس Analyzer (تحلیل‌گر پاسخ)

```
class Analyzer:
```

```
    def __init__(self, t, y, tf):  
        self.t = t  
        self.y = y  
        self.tf = tf  
        self.y_final = tf.K
```

این کلاس را برای استخراج مشخصه‌های پاسخ گذرا طراحی کردم. بردار زمان t ، بردار پاسخ y و تابع تبدیل را دریافت می‌کند. y_final مقدار نهایی پاسخ که برابر بهره سیستم K است.

متد rise_time (محاسبه زمان صعود)

```
def rise_time(self):  
    y10 = 0.1 * self.y_final  
    y90 = 0.9 * self.y_final  
    for i in range(len(self.y)):  
        if t10 is None and self.y[i] >= y10:  
            t10 = self.t[i]  
        if t90 is None and self.y[i] >= y90:  
            t90 = self.t[i]  
    return t90 - t10
```

این متد زمان صعود را محاسبه می‌کند که به مدت زمان لازم برای رسیدن پاسخ از ۱۰٪ به ۹۰٪ مقدار نهایی گفته می‌شود. با پیمایش آرایه پاسخ، اولین لحظه‌ای که خروجی به ۱۰٪ و ۹۰٪ می‌رسد ثبت شده و اختلاف آن‌ها برگردانده می‌شود.

متد percent_overshoot (محاسبه درصد فراجهش)

```
def percent_overshoot(self):  
    ymax = max(self.y)  
    if ymax <= self.y_final * 1.001:
```

```

return 0.0

return (ymax - self.y_final) / self.y_final * 100

```

این متد درصد فرآجهش پاسخ را محاسبه می‌کند که نشان می‌دهد سیستم چقدر بیشتر از مقدار نهایی نوسان کرده است. اگر بیشینه پاسخ تقریباً برابر مقدار نهایی باشد فرآجهش صفر در نظر گرفته می‌شود تا از خطای محاسباتی جلوگیری شود.

متد peak_time (محاسبه زمان اوج)

```

def peak_time(self):
    mp = self.percent_overshoot()
    if mp > 0.1:
        peak_index = np.argmax(self.y)
        return self.t[peak_index]
    return None

```

این متد زمان رسیدن پاسخ به بیشینه مقدار را برمی‌گرداند. فقط در صورتی که فرآجهش قابل‌توجهی وجود داشته باشد (بیشتر از ۰.۱٪) این مقدار محاسبه می‌شود، در غیر این صورت None برمی‌گردد چون سیستم بدون فرآجهش زمان اوج معنادار ندارد.

متد settling_time (محاسبه زمان نشست)

```

def settling_time(self):
    upper = self.y_final * 1.02
    lower = self.y_final * 0.98
    for i in range(len(self.y)):
        if self.y[i] > upper or self.y[i] < lower:
            last_bad = i
    return self.t[last_bad + 1]

```

این متد زمان نشست را محاسبه می‌کند که زمانی است که پاسخ برای همیشه در محدوده $\pm 2\%$ مقدار نهایی باقی می‌ماند. با پیمایش کل آرایه آخرین نقطه‌ای که خارج از این محدوده است پیدا می‌شود و زمان بعد از آن به عنوان زمان نشست برگردانده می‌شود.

متدهای steady_state_error / get_all_specs (خطای پایدار و جمع‌بندی)

```
def steady_state_error(self):  
    return abs(self.y_final - self.y[-1])
```

```
def get_all_specs(self):  
    return {"rise_time": ..., "peak_time": ...,  
            "overshoot": ..., "settling_time": ...,  
            "steady_state_error": ...}
```

متد steady_state_error اختلاف بین مقدار نهایی واقعی و آخرین نقطه محاسبه‌شده پاسخ را برمی‌گرداند که نشان‌دهنده خطای پایدار سیستم است. متد get_all_specs همه مشخصه‌های محاسبه‌شده را در یک دیکشنری جمع می‌کند تا به راحتی در رابط گرافیکی نمایش داده شود.

۵. کلاس Plotter (رسم نمودار)

```
class Plotter:  
    def __init__(self, ax, figure):  
        self.ax = ax  
        self.figure = figure
```

این کلاس را برای جداسازی منطق رسم نمودار از بقیه برنامه طراحی کردم. ارجاع به محور رسم و figure شکل کلی matplotlib را دریافت می‌کند تا بتواند نمودار را رسم و به‌روزرسانی کند.

متد plot رسم کامل نمودار پاسخ

```
def plot(self, t, y, specs, tf):  
    self.ax.clear()  
    self.ax.plot(t, y, 'b-', linewidth=2, label='y(t)')  
    self.ax.axhline(y=K, color='gray', linestyle='--')  
    self.ax.fill_between(t, K*0.98, K*1.02, alpha=0.1, color='green')  
    if tr is not None:  
        self.ax.axvline(x=tr, color='orange', linestyle='--')  
    if tp is not None and mp > 0.1:  
        self.ax.plot(tp, max(y), 'r*', markersize=12)  
    if ts is not None:
```



```

self.ax.axvline(x=ts, color='purple', linestyle='-.')

self.ax.legend(fontsize=9)

self.figure.tight_layout()

```

این متد منحنی پاسخ پله را همراه با تمام مشخصه‌های گذرا رسم می‌کند. ابتدا محور پاک شده و منحنی اصلی آبی رنگ رسم می‌شود. خط خاکستری چین مقدار نهایی و نوار سبز محدوده $\pm 2\%$ را نشان می‌دهد. خطوط عمودی نارنجی، قرمز و بنفش به ترتیب زمان صعود، زمان اوج و زمان نشست را علامت می‌زنند. ستاره قرمز نقطه اوج فراجهش را مشخص می‌کند و در نهایت legend و عنوان نمودار تنظیم می‌شوند.

۶. کلاس App (رابط گرافیکی)

```

class App:

    def __init__(self, root):

        self.root = root

        self.root.title("Step Response Analyzer")

        self.root.geometry("1000x650")

        self.order_var = tk.IntVar(value=2)

```

این کلاس را برای مدیریت کامل رابط گرافیکی طراحی کردم. پنجره اصلی با اندازه ۱۰۰۰ در ۶۵۰ پیکسل تنظیم می‌شود. order_var مرتبه سیستم انتخابی توسط کاربر (۱ یا ۲) را نگه می‌دارد و مقدار پیش‌فرض روی مرتبه دوم تنظیم شده است.

متد on_order_change (تغییر فیلدهای ورودی)

```

def on_order_change(self):

    if self.order_var.get() == 1:

        self.label_zeta.pack_forget()

        self.entry_zeta.pack_forget()

        self.label_tau.pack(anchor="w")

        self.entry_tau.pack(pady=4)

    else:

        self.label_tau.pack_forget()

        self.label_zeta.pack(anchor="w")

        self.label_wn.pack(anchor="w")

```

این متد وقتی کاربر مرتبه سیستم را تغییر می‌دهد فراخوانی می‌شود. اگر مرتبه اول انتخاب شود فیلدهای ζ و ω_n مخفی شده و فقط فیلد τ نمایش داده می‌شود. اگر مرتبه دوم انتخاب شود برعکس این اتفاق می‌افتد. این روش `pack` و `pack_forget` باعث می‌شود رابط کاربری همیشه فقط فیلدهای مرتبط را نشان دهد.

متد `run_analysis` (اجرای تحلیل کامل)

```
def run_analysis(self):
    try:
        K = float(self.entry_K.get())
    except:
        messagebox.showerror("Error", "K is not valid!")
        return
    if order == 1:
        tf = TransferFunction(K, order=1, tau=tau)
    else:
        tf = TransferFunction(K, order=2, zeta=zeta, wn=wn)
    step = StepResponse(tf)
    t, y = step.compute()
    analyzer = Analyzer(t, y, tf)
    specs = analyzer.get_all_specs()
    plotter = Plotter(self.ax, self.figure)
    plotter.plot(t, y, specs, tf)
    self.canvas.draw()
```

این متد قلب اصلی برنامه است و با کلیک روی دکمه `Analyze` اجرا می‌شود. ابتدا مقادیر ورودی از `Entry` ها خوانده و اعتبارسنجی می‌شوند؛ اگر نامعتبر باشند پیام خطا نمایش داده می‌شود. سپس بر اساس مرتبه انتخابی یک `TransferFunction` ساخته می‌شود. به ترتیب `StepResponse` پاسخ را محاسبه می‌کند، `Analyzer` مشخصه‌ها را استخراج می‌کند و `Plotter` نمودار نهایی را رسم می‌کند. در پایان مقادیر عددی نتایج روی `Label` ها نمایش داده می‌شوند.

متد `clear_plot` (پاک کردن نمودار و نتایج)

```
def clear_plot(self):
    self.ax.clear()
    self.ax.set_title("Step Response\n(Enter parameters and click Analyze)")
    self.canvas.draw()
    for lbl in [self.label_tr, ...]:
```

```
lbl.config(text="-")
```

این متد با کلیک روی دکمه Clear اجرا می‌شود. نمودار به حالت اولیه خالی برمی‌گردد و عنوان پیش‌فرض دوباره تنظیم می‌شود. تمام Label های نتایج نیز به علامت خط تیره برگردانده می‌شوند تا برنامه برای تحلیل بعدی آماده باشد.

۷. نقطه ورودی برنامه

```
root = tk.Tk()  
app = App(root)  
root.mainloop()
```

این بلوک نقطه شروع برنامه است. یک پنجره `tkinter` ساخته می‌شود، آن را به `App` می‌دهیم و با `mainloop()` برنامه را در حالت اجرا نگه می‌داریم تا کاربر پنجره را ببندد.